Project Title

jsh - a mini unix shell. Mac computers use zsh (z shell) so I decided to name my project jsh (Joe shell).

Project Purpose

To understand what a shell does behind the scenes with user commands and outputs.

Project Goals

To implement a fully functional shell that can run almost all of the commands that a user would input to other popular shells such as zsh and BASH.

Implemented so far

So far I have implemented both builtin commands as well as executable commands. cd allows users to change directory, ls lists all files and directories in the current directory, pwd lists the current directory, clear clears out the command line and exit allows the user to exit the shell.

CS370 Concepts Used

I used the compilation pipeline we learned in class of generating assembly and object code. I used -arch x86_64 to compile an executable for a CISC type architecture. I used pointers and arrays of strings with variables like allTokens. I used the heap with functions like getline(), and free().

How concepts are reflected in my code

I have used the lldb debugger, objdump and generated assembly file to inspect my code.

```
josephklinck@Josephs-MacBook-Pro jsh % objdump -D main

main:   file format mach-o 64-bit x86-64

Disassembly of section __TEXT,__text:

0000000100000470 <_main>:
100000470: 55                          pushq   %rbp
100000471: 48 89 e5                    movq    %rsp, %rbp
100000474: 48 81 ec 20 01 00 00        subq    $0x120, %rsp            ## imm = 0x120
10000047b: 48 8b 05 86 0b 00 00        movq    0xb86(%rip), %rax       ## 0x100001008 <_wait+0x100
001008>
100000482: 48 8b 00                    movq    (%rax), %rax
100000485: 48 89 45 f8                 movq    %rax, -0x8(%rbp)
100000489: c7 85 2c ff ff ff 00 00 00 00       movl    $0x0, -0xd4(%rbp)
100000493: 89 bd 28 ff ff ff           movl    %edi, -0xd8(%rbp)
100000499: 48 89 b5 20 ff ff ff        movq    %rsi, -0xe0(%rbp)
1000004a0: 48 c7 85 18 ff ff ff 00 00 00 00    movq    $0x0, -0xe8(%rbp)
1000004ab: 48 c7 85 10 ff ff ff 00 00 00 00    movq    $0x0, -0xf0(%rbp)
1000004b6: 48 8d 7d a0                 leaq    -0x60(%rbp), %rdi
1000004ba: 31 f6                       xorl    %esi, %esi
1000004bc: ba 58 00 00 00              movl    $0x58, %edx
1000004c1: e8 42 03 00 00              callq   0x100000808 <_wait+0x100000808>
1000004c6: c7 85 fc fe ff ff 00 00 00 00       movl    $0x0, -0x104(%rbp)
1000004d0: 48 8d bd 30 ff ff ff        leaq    -0xd0(%rbp), %rdi
1000004d7: be 64 00 00 00              movl    $0x64, %esi
1000004dc: e8 1b 03 00 00              callq   0x1000007fc <_wait+0x1000007fc>
1000004e1: 48 89 c6                    movq    %rax, %rsi
1000004e4: 48 8d 3d 41 03 00 00        leaq    0x341(%rip), %rdi       ## 0x10000082c <_wait+0x100
00082c>
1000004eb: b0 00                       movb    $0x0, %al
1000004ed: e8 22 03 00 00              callq   0x100000814 <_wait+0x100000814>
```

Figure 1: objdump disassembly of main

```
josephklinck@Josephs-MacBook-Pro jsh % lldb ./main
(lldb) target create "./main"
Current executable set to '/Users/josephklinck/desktop/CS370/jsh/main' (x86_64).
(lldb) b main
Breakpoint 1: where = main`main, address = 0x0000000100000470
(lldb) r
Process 22404 launched: '/Users/josephklinck/desktop/CS370/jsh/main' (x86_64)
warning: libobjc.A.dylib is being read from process memory. This indicates that LLDB could not rea
d from the host's in-memory shared cache. This will likely reduce debugging performance.

Process 22404 stopped
* thread #1, queue = 'com.apple.main-thread', stop reason = breakpoint 1.1
      frame #0: 0x0000000100000470 main`main
main`main:
->  0x100000470 <+0>:  pushq  %rbp
    0x100000471 <+1>:  movq   %rsp, %rbp
    0x100000474 <+4>:  subq   $0x120, %rsp ; imm = 0x120
    0x10000047b <+11>: movq   0xb86(%rip), %rax ; (void *)0x00007ff8503a4640: __stack_chk_guard
Target 0: (main) stopped.
(lldb) register read rip rbp
     rip = 0x0000000100000470  main`main
     rbp = 0x00000003041404a0
(lldb) n
Process 22404 stopped
* thread #1, queue = 'com.apple.main-thread', stop reason = instruction step over
      frame #0: 0x0000000100000471 main`main + 1
main`main:
->  0x100000471 <+1>:  movq   %rsp, %rbp
    0x100000474 <+4>:  subq   $0x120, %rsp ; imm = 0x120
    0x10000047b <+11>: movq   0xb86(%rip), %rax ; (void *)0x00007ff8503a4640: __stack_chk_guard
    0x100000482 <+18>: movq   (%rax), %rax
Target 0: (main) stopped.
(lldb) register read rip rbp
     rip = 0x0000000100000471  main`main + 1
     rbp = 0x00000003041404a0
(lldb)
```

Figure 2: lldb session stepping through main and reading registers

Testing

I ran the program with all of the commands from my project goals and they are all running correctly.

```
josephklinck@Josephs-MacBook-Pro jsh % ./main
/Users/josephklinck/Desktop/CS370/jsh: cd ..
/Users/josephklinck/Desktop/CS370: ls
CS370 Week 4 Activity Worksheet.pages   cslab
CS370 Week 4 Activity Worksheet.pdf     jsh
cs370-week1-lab                         practice
cs370-week2-lab                         Project_proposal.pdf
cs370-week3-lab
/Users/josephklinck/Desktop/CS370: cd jsh
/Users/josephklinck/Desktop/CS370/jsh: ls
commands.c      main            Makefile        screenShots
commands.h      main.c          README.md
/Users/josephklinck/Desktop/CS370/jsh: pwd
/Users/josephklinck/Desktop/CS370/jsh
/Users/josephklinck/Desktop/CS370/jsh:
```

Figure 3: Running jsh with cd, ls and pwd

Challenges

First I had to learn all about how processes are created. From there I was able to implement cd, pwd and exit. I ran into some issues with clear and ls as I wasn't checking the fork return value and having the parent process (the shell) wait for the child process (ls or clear) to finish. The shell was also crashing (segmentation fault) when I input an empty line, this was solved on line 38 in main.c by checking if allTokens[0] was null. I also ran into issue with my Makefile as I was using spaces instead of tabs.

Future Development

I need to further develop the ability to take other user inputs. While I have ls working right now, it cannot process more complex commands like ls -al. I also need to look at a better way to implement the decision making for user inputs as I am currently just using a set of conditional statements. I also need to correct cd as just running it by itself right now does not go to the users home directory.

CS370 Concepts to be applied next

Later in the course we're going to learn more about processes, signals and system level I/O. I can see how those concepts can help with further improvements upon my project.